$$[\text{Word Indices}] \longrightarrow \mathbf{W_1}\ (\text{Embeddings}) \longrightarrow \mathbf{W}_{\text{conv}}\ (\text{Convolution}) \longrightarrow \text{ReLU} \longrightarrow \text{Pooling} \longrightarrow \mathbf{W_2}\ (\text{Output}) \longrightarrow \text{FINAL 3D VECTOR}$$

$$[\text{Word Indices}] \quad \underbrace{\longrightarrow \mathbf{W_1}}_{\text{FNN / Embedding}} \quad \longrightarrow \underbrace{(\mathbf{W}_{\text{conv}} \longrightarrow \text{ReLU} \longrightarrow \text{Pooling})}_{\textbf{PURE CORE CNN ENGINE}} \quad \underbrace{\longrightarrow \mathbf{W_2}}_{\text{FNN Output}} \quad \longrightarrow \text{FINAL 3D VECTOR}$$

Let's do a complete, step-by-step mathematical walkthrough of how a **1D Convolutional Neural Network (CNN)** processes text using your exact W_1 word embeddings!

We will use a 3-word sentence from our vocabulary to clearly see the **sliding window (kernel)** in action:

- **Sentence:** "python writing programming"
- **Word Indices & Vectors from your W_1 Table (2 Hidden Nodes):**
    1. python (Index 0) = $[0.20, -0.10]$
    2. writing (Index 2) = $[0.35, -0.15]$
    3. programming (Index 1) = $[-0.15, 0.30]$

Step 1: Set Up the CNN Filter (W_{conv})

Unlike an FNN (which adds all words together) or an LSTM (which reads them one by one in a loop), a CNN uses a **sliding filter (kernel)**.

Let's say our **Kernel Size = 2** (meaning the filter looks at **2 adjacent words at a time**, like a 2-word phrase detector).

Since each word vector has 2 numbers (our 2 hidden nodes), our filter needs to match that size. Let's define a clean 2×2 filter matrix (W_{conv}):

$$W_{\text{conv}} = \begin{bmatrix} 0.5 & 0.1 \\ -0.2 & 0.4 \end{bmatrix}$$

Step 2: Slide the Filter Across the Sentence (Convolution)

Our sentence has 3 words, which means our sliding window of size 2 can fit in **two positions**:

Position 1: Looking at Words 1 & 2 ("python" and "writing")

- **Input Window Matrix:**

$$\text{Window}_1 = \begin{bmatrix} \text{python} \\ \text{writing} \end{bmatrix} = \begin{bmatrix} 0.20 & -0.10 \\ 0.35 & -0.15 \end{bmatrix}$$

- **Convolution Operation (Dot Product with Filter):**

We multiply our filter across this window to produce a local phrase feature vector:

- Feature 1: $(0.5 \times 0.20) + (0.1 \times -0.10) + (0.5 \times 0.35) + (0.1 \times -0.15) = 0.10 - 0.01 + 0.175 - 0.015 = \mathbf{0.25}$
- Feature 2: $(-0.2 \times 0.20) + (0.4 \times -0.10) + (-0.2 \times 0.35) + (0.4 \times -0.15) = -0.04 - 0.04 - 0.07 - 0.06 = \mathbf{-0.21}$

- **Raw Feature Vector 1:** $[0.25, -0.21]$

Position 2: Sliding to Words 2 & 3 ("writing" and "programming")

- **Input Window Matrix:**

$$\text{Window}_2 = \begin{bmatrix} \text{writing} \\ \text{programming} \end{bmatrix} = \begin{bmatrix} 0.35 & -0.15 \\ -0.15 & 0.30 \end{bmatrix}$$

- **Convolution Operation (Dot Product with Filter):**
  - Feature 1: $(0.5 \times 0.35) + (0.1 \times -0.15) + (0.5 \times -0.15) + (0.1 \times 0.30) = 0.175 - 0.015 - 0.075 + 0.03 = \mathbf{0.115}$
  - Feature 2: $(-0.2 \times 0.35) + (0.4 \times -0.15) + (-0.2 \times -0.15) + (0.4 \times 0.30) = -0.07 - 0.06 + 0.03 + 0.12 = \mathbf{0.02}$
  - **Raw Feature Vector 2:** $[0.115, 0.02]$

Step 3: Apply ReLU (Activation Function)

Just like in your FNN, we pass both local feature vectors through **ReLU** (which turns any negative numbers into 0):

- **Feature Vector 1:** $\max(0, [0.25, -0.21]) = [\mathbf{0.25}, \mathbf{0.0}]$
- **Feature Vector 2:** $\max(0, [0.115, 0.02]) = [\mathbf{0.115}, \mathbf{0.02}]$

Step 4: Max Pooling (Picking the Strongest Feature)

A sentence can have many sliding windows, but our output layer (W_2) needs a single fixed-size vector for the sentence.

To solve this, a CNN uses **Max Pooling**. It looks across all window positions and picks the **maximum value** for each feature column:

- **For Feature Node 1:** $\max(0.25, 0.115) = \mathbf{0.25}$
- **For Feature Node 2:** $\max(0.0, 0.02) = \mathbf{0.02}$

Final CNN Hidden Vector for the Sentence: $[\mathbf{0.25}, \mathbf{0.02}]$

(This vector represents the strongest local phrase patterns detected anywhere in the sentence by our shared W_{conv} filter!)

Step 5: Projecting to the Final 3D Vector via W_2

Finally, we take our pooled CNN vector [0.25, 0.02] and multiply it by your output weight matrix W_2 (just like we did in your FNN) to get the final 3D vector for the sentence!

Summary of What Just Happened

1. **No Bag-of-Words Sum:** Instead of adding all words together blindly, the CNN kept them in order.
2. **The Sliding Rubber Stamp (W_{conv}):** The 2×2 filter stamped across adjacent pairs (python + writing, then writing + programming) to catch local phrase context.

3. **Max Pooling:** It selected the most prominent feature detected across the sentence and passed it to W_2.

Let's run the exact same **1D Convolutional Neural Network (CNN)** workout for our **Second Sentence** using the **exact same shared filter (W_{conv}),** and then compare the final results side-by-side!

- **Sentence 1:** "python writing programming"
- **Sentence 2:** "programming writing python" (The exact same words in reverse order)

Step 0: Our Shared CNN Filter (W_{conv})

$$W_{\text{conv}} = \begin{bmatrix} 0.5 & 0.1 \\ -0.2 & 0.4 \end{bmatrix}$$

Sentence 2 Walkthrough: "programming" → "writing" → "python"

Word Vectors from your W_1 Table:

1. programming (Index 1) = $[-0.15, 0.30]$
2. writing (Index 2) = $[0.35, -0.15]$
3. python (Index 0) = $[0.20, -0.10]$

Our filter size is 2, so it slides across Sentence 2 in **two positions**:

Position 1: Looking at Words 1 & 2 ("programming" and "writing")

- **Input Window Matrix:**

$$\text{Window}_1 = \begin{bmatrix} -0.15 & 0.30 \\ 0.35 & -0.15 \end{bmatrix}$$

- **Convolution Operation (Dot Product with Filter):**
  - Feature 1: $(0.5 \times -0.15) + (0.1 \times 0.30) + (0.5 \times 0.35) + (0.1 \times -0.15) = -0.075 + 0.03 + 0.175 - 0.015 = \mathbf{0.115}$
  - Feature 2: $(-0.2 \times -0.15) + (0.4 \times 0.30) + (-0.2 \times 0.35) + (0.4 \times -0.15) = 0.03 + 0.12 - 0.07 - 0.06 = \mathbf{0.02}$
  - **Raw Feature Vector 1:** $[0.115, 0.02]$

Position 2: Sliding to Words 2 & 3 ("writing" and "python")

- **Input Window Matrix:**

$$\text{Window}_2 = \begin{bmatrix} 0.35 & -0.15 \\ 0.20 & -0.10 \end{bmatrix}$$

- **Convolution Operation (Dot Product with Filter):**

- Feature 1: $(0.5 \times 0.35) + (0.1 \times -0.15) + (0.5 \times 0.20) + (0.1 \times -0.10) = 0.175 - 0.015 + 0.10 - 0.01 = \mathbf{0.25}$
- Feature 2: $(-0.2 \times 0.35) + (0.4 \times -0.15) + (-0.2 \times 0.20) + (0.4 \times -0.10) = -0.07 - 0.06 - 0.04 - 0.04 = \mathbf{-0.21}$
- **Raw Feature Vector 2:** $[0.25, -0.21]$

Step 3: Apply ReLU (Activation Function)

Passing both feature vectors through ReLU (turning negative numbers to 0):

- **Feature Vector 1:** $\max(0, [0.115, 0.02]) = [\mathbf{0.115}, \mathbf{0.02}]$
- **Feature Vector 2:** $\max(0, [0.25, -0.21]) = [\mathbf{0.25}, \mathbf{0.0}]$

Step 4: Max Pooling (Picking the Strongest Feature Across Windows)

We look across both windows to find the maximum value for each feature node:

- **For Feature Node 1:** $\max(0.115, 0.25) = \mathbf{0.25}$
- **For Feature Node 2:** $\max(0.02, 0.0) = \mathbf{0.02}$

Final CNN Hidden Vector for Sentence 2: $[\mathbf{0.25}, \mathbf{0.02}]$

Final Comparison: Sentence 1 vs. Sentence 2 Vector

- **Sentence 1 Vector ("python writing programming"):** $[\mathbf{0.25}, \mathbf{0.02}]$
- **Sentence 2 Vector ("programming writing python"):** $[\mathbf{0.25}, \mathbf{0.02}]$

💡 **A Crucial Discovery About CNNs!**

Look at what just happened: Even though Sentence 1 and Sentence 2 had the words in a different order, **the final CNN output vector was the same.**

Let's run the exact same **1D Convolutional Neural Network (CNN)** workout for **Sentence 3** using the exact same shared filter (W_{conv}), and then compare all three sentences side-by-side!

- **Sentence 1:** "python writing programming" → Vector: [0.25, 0.02]
- **Sentence 2:** "programming writing python" → Vector: [0.25, 0.02]
- **Sentence 3:** "programming python writing" → Let's calculate this one!

Step 0: Our Shared CNN Filter (W_{conv})

$$W_{\text{conv}} = \begin{bmatrix} 0.5 & 0.1 \\ -0.2 & 0.4 \end{bmatrix}$$

Sentence 3 Walkthrough: "programming" $\rightarrow$ "python" $\rightarrow$ "writing"

Word Vectors from your W_1 Table:

1. programming (Index 1) = $[-0.15, 0.30]$
2. python (Index 0) = $[0.20, -0.10]$
3. writing (Index 2) = $[0.35, -0.15]$

Our filter size is 2, so it slides across Sentence 3 in **two positions**:

Position 1: Looking at Words 1 & 2 ("programming" and "python")

- **Input Window Matrix:**

$$\text{Window}_1 = \begin{bmatrix} -0.15 & 0.30 \\ 0.20 & -0.10 \end{bmatrix}$$

- **Convolution Operation (Dot Product with Filter):**
  - Feature 1: $(0.5 \times -0.15) + (0.1 \times 0.30) + (0.5 \times 0.20) + (0.1 \times -0.10) = -0.075 + 0.03 + 0.10 - 0.01 = \mathbf{0.045}$
  - Feature 2: $(-0.2 \times -0.15) + (0.4 \times 0.30) + (-0.2 \times 0.20) + (0.4 \times -0.10) = 0.03 + 0.12 - 0.04 - 0.04 = \mathbf{0.07}$
  - **Raw Feature Vector 1:** $[0.045, 0.07]$

Position 2: Looking at Words 2 & 3 ("python" and "writing")

- *Notice:* This is the exact same window of words we had in Sentence 1, Position 1 (python + writing)!
- **Raw Feature Vector 2:** $[0.25, -0.21]$

Step 3: Apply ReLU (Activation Function)

Passing both feature vectors through ReLU (turning negative numbers to 0):

- **Feature Vector 1:** $\max(0, [0.045, 0.07]) = [\mathbf{0.045}, \mathbf{0.07}]$
- **Feature Vector 2:** $\max(0, [0.25, -0.21]) = [\mathbf{0.25}, \mathbf{0.0}]$

Step 4: Max Pooling (Picking the Strongest Feature Across Windows)

We look across both windows to find the maximum value for each feature node:

- **For Feature Node 1:** $\max(0.045, 0.25) = \mathbf{0.25}$
- **For Feature Node 2:** $\max(0.07, 0.0) = \mathbf{0.07}$

Final CNN Hidden Vector for Sentence 3: $[\mathbf{0.25}, \mathbf{0.07}]$

Final Comparison: All Three Sentences

- **Sentence 1** ("python writing programming"): $[\mathbf{0.25}, \mathbf{0.02}]$
- **Sentence 2** ("programming writing python"): $[\mathbf{0.25}, \mathbf{0.02}]$
- **Sentence 3** ("programming python writing"): $[\mathbf{0.25}, \mathbf{0.07}]$

What This Final Comparison Teaches Us About CNNs

1. **Local Order is Kept:** Within each sliding window, the CNN successfully processed the words in order. For example, "python" followed by "writing" produced a distinct feature vector compared to "programming" followed by "python".
2. **Global Order Can Shift (Max Pooling Effect):** Because Sentence 3 placed "python" and "writing" together in the second window, its Max Pooling picked up a slightly higher value (0.07 vs 0.02) for the second node.
3. **The Core Takeaway:** A CNN is fantastic at finding *what phrases exist* anywhere in a sentence, but because **Max Pooling** collapses the sequence down to just the highest peaks, it still struggles to maintain the strict, global chronological order of the whole sentence (which is why LSTMs and Transformers were needed!).